
APLICACIONES WEB

Edición ampliada con historia, fundamentos detallados, prácticas paso a paso y buenas prácticas profesionales

Libro de texto académico

Parte 1 Ampliada — Semanas 1 a 9

Unidad I: *Fundamentos y Diseño Web*

Unidad II: *Herramientas de Programación Web del Servidor*

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Quinto nivel — Período Académico 2026-2027 PPA

Autor docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Quevedo, Los Ríos — Ecuador

Mayo de 2026

APLICACIONES WEB — Parte 1 Ampliada (Semanas 1 a 9)

Universidad Técnica Estatal de Quevedo, 2026.

Facultad de Ciencias de la Computación.

Carrera de Ingeniería de Software (Rediseño).

Esta edición ampliada incorpora un tratamiento profundo de la historia del desarrollo web, fundamentos teóricos extensos, ejercicios paso a paso plenamente reproducibles en IntelliJ IDEA Ultimate, tablas comparativas de versiones de tecnologías y secciones dedicadas a buenas prácticas profesionales para cada tecnología abordada. Su contenido se ha alineado con el SWEBOK Guide v4.0 [61], las directrices del W3C [66], la guía OWASP Top 10:2021 [48] y las especificaciones HTML Living Standard del WHATWG.

Uso académico no comercial. Se permite la reproducción total o parcial citando la fuente.

Prefacio a la edición ampliada

La presente edición ampliada de la Parte 1 del libro *Aplicaciones Web* responde a una reflexión pedagógica concreta: el desarrollo web moderno no se domina memorizando sintaxis, se domina comprendiendo el **por qué histórico** de cada decisión técnica, las **razones científicas** detrás de cada práctica y los **camino prácticos** que llevan del concepto al artefacto funcional.

A diferencia de la edición original, esta ampliación incorpora cuatro elementos pedagógicos sistemáticos en cada capítulo:

1. **Cajas de historia** que sitúan cada tecnología en su contexto histórico, mencionando autores, fechas, hitos y motivaciones originales que rara vez aparecen en los manuales técnicos pero que resultan indispensables para comprender por qué las cosas son como son.
2. **Tablas de versiones** que documentan la evolución de cada tecnología con las mejoras sustantivas que cada revisión aportó.
3. **Cajas de buenas prácticas** que destilan en pautas accionables el conocimiento profesional acumulado por la industria, frecuentemente disperso en blogs y discusiones técnicas.
4. **Ejercicios paso a paso ampliados** con instrucciones tan detalladas que cualquier estudiante con IntelliJ IDEA Ultimate puede reproducir el resultado exacto descrito en el texto. Cada paso explica *qué* se hace, *por qué* se hace y *cómo verificar* que el resultado es correcto.

La elección de IntelliJ IDEA Ultimate como entorno de referencia obedece a su soporte nativo de primera clase para HTML5, CSS3, JavaScript moderno (ES2024), Sass, PHP, Perl, Java, Spring Boot y los servidores Tomcat/Apache; a la integración nativa de Git, Maven, Docker y bases de datos relacionales sin necesidad de plugins externos; y a la disponibilidad de licencia educativa gratuita para estudiantes universitarios.

El autor docente.

Quevedo, mayo de 2026.

Índice general

Prefacio a la edición ampliada	3
1. Encuadre académico y fundamentos de las aplicaciones web	12
1.1. Encuadre académico de la asignatura	12
1.1.1. Ubicación curricular y prerequisites	12
1.1.2. Articulación con el SWEBOK Guide v4.0	13
1.1.3. Competencias por desarrollar	13
1.1.4. Sistema de evaluación	14
1.2. Historia de las aplicaciones web	14
1.2.1. Las tres eras de la web	14
1.2.2. Evolución temporal de los hitos web	16
1.3. Fundamentos de las aplicaciones web	17
1.3.1. Definición y clasificación de aplicaciones web	18
1.3.2. Características de las aplicaciones web modernas	19
1.3.3. Arquitectura cliente-servidor en aplicaciones web	19
1.3.4. Ventajas y desventajas de las aplicaciones web	22
1.4. Diseño web inclusivo y accesibilidad	23
1.4.1. ¿Qué es la accesibilidad web?	23
1.4.2. Las cuatro categorías de discapacidad relevantes para la web	24
1.4.3. Las Web Content Accessibility Guidelines 2.1 (WCAG 2.1)	25
1.4.4. Criterios de éxito clave de WCAG 2.1	27
1.5. Configuración del entorno con IntelliJ IDEA Ultimate	29
1.6. Buenas prácticas profesionales para HTML semántico	32
1.7. Tabla de referencia rápida: tags HTML5 más usados, atributos y valores	33
1.7.1. Atributos globales (aplican a casi todos los tags)	35
1.7.2. Tags estructurales y de contenido	35
2. HTML y HTML5 para el desarrollo web	37
2.1. Historia y evolución de HTML	37
2.2. Fundamentos de HTML	38
2.2.1. Estructura mínima de un documento HTML5 válido	38
2.2.2. Las cinco categorías de contenido en HTML5	41
2.2.3. Modelo de contenido y reglas de anidamiento	42

2.3.	Elementos semánticos de HTML5 en profundidad	43
2.3.1.	Catálogo de elementos semánticos	43
2.3.2.	¿<article> o <section>? Guía decisional con ejemplo real	43
2.4.	Formularios HTML5 y validación nativa	45
2.4.1.	Tipos de input modernos	45
2.4.2.	Atributos de validación	45
2.5.	Multimedia y APIs de HTML5	47
2.5.1.	La etiqueta <video> con accesibilidad completa	48
2.5.2.	El elemento <picture> para imágenes responsivas	49
2.6.	XML y familias de lenguajes de marcado	50
2.6.1.	Subdialectos de XML relevantes en aplicaciones web	50
2.6.2.	Formatos de intercambio de datos: XML, JSON, YAML, TOML y otros	51
2.7.	Buenas prácticas profesionales en HTML5	52
2.8.	Ejercicios	53
3.	CSS para el diseño y la presentación web	56
3.1.	Historia y evolución de CSS	56
3.1.1.	Tabla evolutiva de CSS	56
3.2.	Fundamentos de CSS	57
3.2.1.	Las tres formas de aplicar estilos	57
3.2.2.	Selectores principales	57
3.2.3.	Pseudo-clases CSS para feedback de validación	58
3.3.	Sistemas modernos de layout	58
3.3.1.	Flexbox: layout unidimensional	59
3.3.2.	CSS Grid: layout bidimensional	61
3.4.	Sass: el preprocesador profesional	61
3.5.	Ejercicios	62
3.6.	Buenas prácticas CSS profesional	62
4.	JavaScript moderno y programación asíncrona	63
4.1.	Historia de JavaScript	63
4.1.1.	Tabla evolutiva de ECMAScript	63
4.2.	Fundamentos modernos	64
4.2.1.	Variables: por qué let/const y nunca var	64
4.2.2.	Funciones flecha y funciones regulares	64
4.2.3.	Destructuring y spread	65
4.3.	Manipulación del DOM	65
4.4.	Programación asíncrona: fetch, Promises, async/await	67
4.5.	Ejercicios	68
4.6.	Buenas prácticas JavaScript	68
5.	Introducción a la programación del lado del servidor: PERL y PHP	69
5.1.	Programación del lado del servidor: conceptos	69

5.1.1.	Modelo request-response	69
5.1.2.	Arquitectura: navegador → Apache/Nginx → intérprete	69
5.2.	PERL para CGI: el pionero	70
5.2.1.	Hola mundo en PERL+CGI	70
5.3.	PHP: el lenguaje dominante de la web tradicional	71
5.3.1.	Tabla evolutiva de PHP	71
5.3.2.	Sintaxis básica	71
5.3.3.	Procesamiento de formularios	72
5.4.	Ejercicios	74
5.5.	Buenas prácticas PHP	75
6.	Java para aplicaciones web: JSP, Servlets y Spring Boot	76
6.1.	Historia de Java en la web	76
6.1.1.	Tabla evolutiva	76
6.2.	Servlets y JSP: el modelo histórico	77
6.3.	Spring Boot: el estándar moderno	78
6.4.	Spring Boot con JPA y BD H2	79
6.5.	Ejercicios	82
6.6.	Buenas prácticas Spring Boot	82
7.	ASP.NET Core: la plataforma .NET para aplicaciones web modernas	83
7.1.	Historia de ASP.NET	83
7.2.	Conceptos fundamentales	83
7.2.1.	La línea de comandos <code>dotnet</code>	84
7.2.2.	Estructura típica de un proyecto	84
7.3.	Ejemplo completo: Web API CRUD	84
7.4.	Razor Pages y MVC tradicional	88
7.5.	Ejercicios	89
7.6.	Buenas prácticas ASP.NET Core	89
8.	Tutoría de refuerzo: consolidación y resolución de blockers	90
8.1.	Mapa conceptual del primer corte	90
8.2.	Errores frecuentes detectados (sem 1–7)	91
8.3.	Ejercicios integradores	91
8.4.	Repaso intensivo de los temas más difíciles	92
8.4.1.	Tema 1: Diferencia GET vs POST	92
8.4.2.	Tema 2: Validación cliente vs servidor	93
8.4.3.	Tema 3: Codificación de caracteres UTF-8	93
8.5.	Preparación para la EP1	93
9.	Seguridad en aplicaciones web; Evaluación Parcial Primer Corte	94
9.1.	Historia del OWASP y la cultura de seguridad web	94
9.2.	OWASP Top 10:2021 [48]	94
9.3.	Defensas técnicas básicas	95

9.3.1. Anti-SQL Injection: prepared statements	95
9.3.2. Anti-XSS: escape de salida	96
9.3.3. Cookies de sesión seguras	97
9.3.4. Hash de contraseñas con BCrypt	97
9.3.5. Cabeceras de seguridad HTTP	98
9.4. Ejercicios	98
9.5. Evaluación Parcial Primer Corte	99
9.5.1. Estructura	99
9.5.2. Temas garantizados	99
9.6. Buenas prácticas de seguridad	100
10. Gestión de estado y objetos integrados en aplicaciones web	101
10.1. Introducción: el dilema fundamental de HTTP	101
10.1.1. Las cinco estrategias de gestión de estado	102
10.1.2. Los flags de seguridad de cookies en 2026	102
10.2. El objeto Request: lectura segura de la petición	103
10.2.1. Componentes del objeto Request	103
10.2.2. Lectura del Request en Java Servlets / Spring Boot	105
10.2.3. Lectura del Request en ASP.NET Core 8	107
10.3. El objeto Response: control total de la respuesta HTTP	108
10.3.1. Códigos de estado HTTP — la guía exhaustiva	108
10.3.2. Errores estandarizados con RFC 7807 ProblemDetails	108
10.3.3. Cabeceras de seguridad obligatorias en producción	109
10.4. El objeto Session: estado entre peticiones	110
10.4.1. Mecánica detallada de las sesiones de servidor	110
10.4.2. Implementación comparada en las tres plataformas	110
10.4.3. Sesiones en PHP 8.3 con configuración profesional	111
10.5. Almacenamiento del cliente: localStorage, sessionStorage e IndexedDB	115
10.5.1. Comparativa de las APIs de almacenamiento del cliente	115
10.5.2. Ejemplos prácticos de almacenamiento	115
10.6. Práctica integrada: carrito de compras con sesiones	117
10.7. Buenas prácticas profesionales en gestión de estado	121
10.8. Ejercicio resuelto adicional con resultado visual	121
11. Acceso a datos desde aplicaciones web: SQL parametrizado y ORM	124
11.1. Historia y panorama del acceso a datos	124
11.1.1. Las cuatro estrategias de acceso a datos	125
11.1.2. Tabla evolutiva de los ORM principales	125
11.2. Consultas SQL desde aplicaciones web	125
11.2.1. La amenaza fundamental: SQL Injection	125
11.2.2. La defensa: sentencias preparadas en las tres plataformas	126
11.3. El patrón Repository: encapsular el acceso a datos	133
11.3.1. Repositorio típico en Spring Data JPA	133

11.4. El problema N+1 y su solución	134
11.4.1. Solución en cada ORM	134
11.5. Migraciones de esquema con Flyway y Liquibase	135
11.6. Procedimientos almacenados: cuándo aún tiene sentido	137
11.7. Práctica integrada: CRUD ORM vs SQL puro	137
11.8. Buenas prácticas profesionales en acceso a datos	143
11.9. Ejercicio resuelto adicional con resultado visual	143
12. Patrones de arquitectura para aplicaciones web	147
12.1. Principios fundacionales de arquitectura de software	147
12.1.1. Los principios SOLID aplicados a la arquitectura	147
12.1.2. Atributos de calidad ISO/IEC 25010	148
12.2. Patrón 1: Arquitectura por capas (3-capas y 4-capas)	148
12.2.1. Estructura clásica	148
12.2.2. Implementación canónica en Spring Boot	148
12.2.3. Variante: Arquitectura Hexagonal (Ports and Adapters)	149
12.3. Patrón 2: Arquitectura Orientada a Servicios (SOA)	150
12.4. Patrón 3: Microservicios	150
12.4.1. Comparativa SOA vs microservicios	150
12.4.2. Cuándo NO usar microservicios	151
12.4.3. Patrones esenciales de microservicios	151
12.5. Patrón 4: Arquitectura dirigida por eventos (EDA)	151
12.6. Patrón 5: Peer-to-peer (P2P)	152
12.7. Documentación arquitectónica con el modelo C4 de Brown	152
12.7.1. Los cuatro niveles del modelo C4	152
12.7.2. Ejemplo: diagrama de contexto del PFC	152
12.7.3. Ejemplo: diagrama de contenedores del PFC	153
12.8. Architectural Decision Records (ADR)	153
12.9. Práctica integrada: documentar el PFC con C4 y ADRs	154
12.10. Buenas prácticas profesionales en arquitectura	157
12.11. Ejercicio resuelto adicional con resultado visual	158
13. Diseño de arquitecturas web escalables	160
13.1. Escalabilidad en aplicaciones web: vertical vs horizontal	160
13.1.1. Definiciones formales	160
13.1.2. El teorema CAP de Brewer	160
13.2. Estrategias de balanceo de carga	161
13.2.1. Algoritmos de balanceo	161
13.2.2. Implementaciones populares	162
13.3. Patrones de caché en aplicaciones web	163
13.3.1. Los cinco patrones canónicos de caché	163
13.3.2. Cache-aside con Redis: el patrón dominante	163
13.3.3. Estrategias de invalidación	165

13.4. Patrones avanzados: CDN, Edge caching y read replicas	166
13.4.1. CDN para contenido estático	166
13.4.2. Réplicas de lectura para BD	166
13.5. Métodos de evaluación arquitectónica	166
13.5.1. ATAM (Architecture Tradeoff Analysis Method)	166
13.5.2. Lightweight ATAM para equipos pequeños	166
13.6. Práctica integrada: implementar cache-aside con benchmark	167
13.7. Sumativa de la semana 13: Exposición Frameworks Backend	169
13.8. Buenas prácticas profesionales en escalabilidad	169
13.9. Contraste bibliográfico de los conceptos clave	170
13.10Ejercicio resuelto adicional con resultado visual	170
14.El patrón Modelo-Vista-Controlador en aplicaciones web modernas	173
14.1. Historia del patrón MVC	173
14.1.1. Tabla evolutiva del MVC en frameworks web	174
14.2. Anatomía del patrón MVC en la web	174
14.2.1. Adaptación del MVC original al contexto HTTP	174
14.2.2. Las tres capas en detalle	174
14.3. Ciclo de vida de una petición MVC en Spring Boot	175
14.4. Server-side MVC vs SPA + API REST	176
14.4.1. ¿Cuál usar en el PFC?	176
14.5. Implementación canónica: MVC server-side completo	177
14.6. Variantes del MVC	184
14.6.1. MVP (Model-View-Presenter)	184
14.6.2. MVVM (Model-View-ViewModel)	184
14.6.3. Flux y Redux	184
14.6.4. Signals (Angular 17+, SolidJS)	184
14.7. Práctica integrada: CRUD MVC con debugging	184
14.8. Buenas prácticas profesionales en MVC	188
14.9. Contraste bibliográfico de los conceptos clave	188
14.10Ejercicio resuelto adicional con resultado visual	188
15.Servicios web: arquitecturas REST, SOAP y diseño de APIs	191
15.1. Historia de los servicios web	191
15.1.1. Tabla evolutiva de los servicios web	192
15.2. Las seis restricciones de la arquitectura REST	192
15.2.1. Modelo de Madurez de Richardson	192
15.3. Diseño de URIs y verbos HTTP	193
15.3.1. Convenciones de nomenclatura	193
15.3.2. Mapeo verbo-acción canónico	194
15.3.3. Idempotencia y seguridad	194
15.4. Implementación de APIs REST en las tres plataformas	194
15.4.1. Spring Boot 3.4 con Spring Web	194

15.4.2. ASP.NET Core 8	198
15.4.3. PHP 8.3 con Slim Framework 4	200
15.5. REST vs SOAP: la comparativa definitiva	201
15.5.1. Cuándo aún tiene sentido SOAP en 2026	202
15.5.2. Ejemplo SOAP mínimo	202
15.6. Documentación con OpenAPI 3.0 (Swagger)	203
15.6.1. ¿Qué es OpenAPI?	203
15.6.2. Configurar OpenAPI en Spring Boot	204
15.7. Autenticación de APIs con JWT	204
15.7.1. Anatomía de un JWT	205
15.7.2. Flags estándar del payload (<i>claims</i>)	205
15.7.3. JWT en el PFC: configuración Spring Security	205
15.8. Práctica integrada: API REST consumiendo servicio externo	207
15.9. Práctica integrada: PFC Entrega 2 (Semana 15)	215
15.10 Buenas prácticas profesionales en APIs REST	215
15.11 Contraste bibliográfico de los conceptos clave	216
15.12 Ejercicio resuelto adicional con resultado visual	216
16. Entrega final del PFC: pruebas, cobertura, seguridad y publicación	219
16.1. Pruebas en aplicaciones web modernas	219
16.1.1. Niveles de pruebas en aplicaciones web	219
16.1.2. La pirámide de pruebas en detalle	220
16.2. Pruebas unitarias con JUnit 5 y Mockito	220
16.3. Pruebas de integración con MockMvc y Testcontainers	222
16.4. Cobertura con JaCoCo	224
16.5. Pruebas de carga con k6	225
16.6. Auditoría de frontend con Lighthouse	227
16.6.1. Las cuatro métricas core	227
16.6.2. Cómo ejecutar Lighthouse	227
16.6.3. CLI alternativa para CI/CD	227
16.7. Auditoría de seguridad: 6 controles OWASP mínimos	228
16.8. Práctica integrada: PFC Entrega Final v1.0.0	228
16.9. Buenas prácticas profesionales en pruebas y entregas	229
16.10 Contraste bibliográfico de los conceptos clave	230
16.11 Ejercicio resuelto adicional con resultado visual	230
17. Defensa del PFC y Evaluación Parcial Segundo Corte	233
17.1. Cómo preparar la defensa oral del PFC	233
17.1.1. Estructura recomendada de la presentación	233
17.1.2. Las 20 preguntas más frecuentes del tribunal	234
17.1.3. Errores comunes en defensas anteriores	234
17.2. Estructura de la Evaluación Parcial Segundo Corte	235
17.2.1. Temas garantizados en la EP2	235

17.2.2. Ejemplos del tipo de preguntas que aparecen	236
17.3. Preparación efectiva para la EP2	237
17.3.1. Plan de estudio de 7 días previos al examen	237
17.3.2. Estrategia durante el examen	237
17.4. Reflexión sobre el segundo corte	238
17.5. Contraste bibliográfico de los conceptos clave de la EP2	238
17.6. Ejercicio resuelto adicional con resultado visual	238
18. Tutoría de refuerzo final y cierre del curso	240
18.1. Mapa integrador de la asignatura	240
18.1.1. Las cuatro unidades como una historia coherente	240
18.1.2. Tabla resumen de tecnologías dominadas	241
18.2. Errores comunes detectados en las semanas 10–17	241
18.3. Repaso intensivo de los temas más difíciles	242
18.3.1. Tema 1: Diferencia REST vs SOAP	242
18.3.2. Tema 2: JWT — estructura y uso correcto	242
18.3.3. Tema 3: ORM y problema N+1	242
18.3.4. Tema 4: Cache-aside con Redis	243
18.4. Banco de ejercicios de consolidación	243
18.5. Realimentación individual del PFC	244
18.5.1. Criterios de excelencia observados	244
18.5.2. Mejoras frecuentemente sugeridas	244
18.6. El curso en perspectiva profesional	244
18.6.1. Cómo se ve el desarrollo web profesional en 2026	245
18.6.2. Próximos pasos en la formación	245
18.6.3. Certificaciones que aportan valor profesional en 2026	245
18.7. Conclusión del curso	245
18.8. Cierre administrativo de la asignatura	246

Encuadre académico y fundamentos de las aplicaciones web

La web no es un medio: es un ecosistema. Comprender sus fundamentos es comprender la infraestructura digital de la civilización contemporánea.

— Tim Berners-Lee, inventor de la World Wide Web

Resultado de aprendizaje de la semana

Al concluir la semana 1, el estudiante construye interfaces web semánticas, accesibles y responsivas **en su nivel introductorio**, integrando HTML5, CSS3 y JavaScript para crear aplicaciones del lado del cliente que cumplan con los estándares del W3C y los principios del diseño web inclusivo (WCAG 2.1) [66].

1.1 Encuadre académico de la asignatura

Antes de adentrarnos en los aspectos técnicos del desarrollo web, conviene situar la asignatura dentro del plan de estudios y aclarar los acuerdos que regirán el trabajo durante las dieciocho semanas. Esta sección presenta tres aspectos institucionales fundamentales: la ubicación curricular de la materia y los conocimientos previos que se asumen; las modalidades formales de evaluación según la norma académica de la UTEQ; y las políticas profesionales que se aplicarán en clase (asistencia, entregas, integridad académica). La claridad sobre estos acuerdos al inicio del curso evita malentendidos posteriores y permite al estudiante planificar su esfuerzo con realismo.

1.1.1 Ubicación curricular y prerrequisitos

La asignatura Aplicaciones Web pertenece al campo de estudio de *Diseño de Software (DES.dd)* en la Carrera de Ingeniería de Software, ubicada en el quinto período académico de la malla curricular vigente dentro de la Unidad de Organización Curricular Profesional. Posee tres créditos académicos distribuidos en 144 horas totales, de las cuales 48 corresponden a clases presenciales

teóricas y prácticas, 48 al aprendizaje autónomo y 48 al contacto directo con el docente.

Sus prerequisites formales son *Base de Datos* y *Estructura de Datos*, ambos de tercer nivel. El primero garantiza que el estudiante maneje el modelo relacional y el lenguaje SQL como insumo indispensable para el acceso a datos del bloque servidor; el segundo asegura el dominio de listas, árboles, mapas hash y la complejidad algorítmica $O(n \log n)$ y $O(n^2)$, elementos sin los cuales no es posible diseñar APIs eficientes ni razonar sobre escalabilidad. La ausencia de cualquiera de estos prerequisites compromete severamente el aprovechamiento del curso.

1.1.2 Articulación con el SWEBOK Guide v4.0

El *Software Engineering Body of Knowledge* en su cuarta versión [61] reconoce explícitamente al desarrollo web como una de las áreas transversales de la práctica profesional. La asignatura articula los siguientes núcleos del SWEBOK: requisitos de software (KA02), diseño de software (KA03), construcción de software (KA04), calidad del software (KA10) y seguridad de software (KA15). El estudiante no construye únicamente código que *¡funciona!*; construye software que satisface estándares internacionales de calidad, seguridad y accesibilidad.

1.1.3 Competencias por desarrollar

Esta asignatura forma al estudiante en cuatro frentes complementarios: la **capacidad técnica** de implementar interfaces y servicios web modernos; el **razonamiento arquitectónico** para tomar decisiones de diseño justificadas; la **conciencia de calidad** (seguridad, rendimiento, accesibilidad); y la **disciplina profesional** (versionado, pruebas, documentación). La caja siguiente recoge la formulación oficial de la competencia general.

Competencia general de la asignatura

Analizar, diseñar, implementar y evaluar aplicaciones web mediante tecnologías de diseño, lenguajes de programación cliente y servidor, patrones de arquitectura y servicios web, aplicando estándares de calidad, accesibilidad e inclusión, con el fin de construir *soluciones informáticas robustas, escalables y pertinentes* que respondan a las necesidades productivas, sociales y tecnológicas del Ecuador, de la región y del mundo.

Esta competencia general se descompone en cuatro resultados de aprendizaje unitarios (RAU):

- RAU 1.** Construir interfaces web semánticas, accesibles y responsivas con HTML5, CSS3 y JavaScript siguiendo W3C y WCAG 2.1.
- RAU 2.** Desarrollar aplicaciones web dinámicas del lado del servidor con PERL, PHP, ASP.NET y Java/JSP, implementando lógica de negocio, procesamiento de formularios y mecanismos de seguridad.
- RAU 3.** Diseñar e implementar soluciones web que integren gestión de estado, acceso a datos relacionales mediante SQL parametrizado y ORM, y patrones arquitectónicos escalables (multicapa, SOA, EDA, P2P).

RAU 4. Construir una aplicación web completa bajo el patrón MVC con un *framework* moderno, exponiendo e integrando servicios web REST y SOAP, y aplicando criterios de calidad, seguridad e interoperabilidad.

1.1.4 Sistema de evaluación

La calificación final se construye a partir de tres tipos de actividades ponderadas según la Norma Académica de la UTEQ: *Gestión del Aprendizaje (GA)*, *Tareas y Proyectos Autónomos (TA)* y *Evaluaciones (EV)*. La asignatura organiza dos cortes evaluativos (semanas 9 y 17) y un Proyecto Fin de Curso entregado en tres hitos parciales (semanas 5, 9 y 12) y un sistema final completo en la semana 16.

1.2 Historia de las aplicaciones web

Comprender la web actual exige conocer su origen: las decisiones arquitectónicas tomadas en los años noventa (HTTP sin estado, URLs opacas, hipertexto distribuido) condicionan todavía hoy la forma en que diseñamos aplicaciones. La siguiente caja presenta los hitos fundacionales que conviene tener presente como contexto histórico permanente.

Los orígenes — del hipertexto a la World Wide Web

La idea del hipertexto precede a la web por décadas. Vannevar Bush concibió en 1945 el sistema *Memex* en el artículo *¿As We May Think?* publicado en *The Atlantic Monthly*, una máquina capaz de almacenar libros y vincularlos mediante asociaciones siguiendo el modo en que opera la mente humana [10]. Ted Nelson acuñó el término *hipertexto* en 1965 dentro de su proyecto Xanadu, que aspiraba a crear un repositorio universal de documentos enlazados [38].

El paso decisivo lo dio Tim Berners-Lee en marzo de 1989 cuando, en el laboratorio del CERN en Ginebra, presentó el documento titulado *Information Management: A Proposal* [5]. Su intención era resolver un problema interno: la fuga de información cuando los investigadores abandonaban el centro. La propuesta combinó tres ingredientes existentes (el hipertexto, los protocolos de internet y el sistema de nombres de dominio) en un sistema nuevo: la World Wide Web.

A finales de 1990, Berners-Lee había desarrollado en una computadora NeXT los tres pilares fundacionales de la web: el primer servidor (CERN `httpd`), el primer navegador (WorldWideWeb, luego renombrado Nexus para evitar confusión con la red) y el primer documento HTML. El 6 de agosto de 1991 se publicó el primer sitio web público, alojado en `info.cern.ch`, que aún puede visitarse en su versión preservada en <http://info.cern.ch/hypertext/WWW/TheProject.html> [11].

1.2.1 Las tres eras de la web

La World Wide Web no es un fenómeno monolítico: ha atravesado tres grandes eras evolutivas que reflejan tanto cambios tecnológicos como transformaciones sociales profundas en cómo las

personas interactúan con la información digital. Comprender estas eras es indispensable para situar el desarrollo web contemporáneo dentro de una trayectoria histórica significativa, y para anticipar hacia dónde se dirige.

Cada era se caracteriza por una combinación distintiva de: *(i)* el modelo de participación dominante (quién produce y quién consume el contenido), *(ii)* las tecnologías facilitadoras que hicieron posible esa participación, *(iii)* los modelos de negocio que emergieron y *(iv)* las consecuencias sociales y políticas observables. Berners-Lee, en su revisión retrospectiva de 2019, describió esta evolución como un *indoloroso aprendizaje* porque ninguna era ha estado exenta de patologías propias: la Web 1.0 fue limitada en participación, la Web 2.0 concentró poder en plataformas, y la Web 3.0 enfrenta el desafío de la descentralización genuina [7].

Web 1.0 — la web de los documentos (1989–2004)

Esta primera era se caracterizó por el modelo *de solo lectura* (*read-only web*): una minoría de autores con conocimientos técnicos publicaba contenido, y la mayoría lo consumía pasivamente. Las páginas eran fundamentalmente estáticas, escritas a mano en HTML sin interactividad significativa. O'Reilly y Battelle [44] caracterizaron esta era como la del *úsitio web folletoz*: una transposición digital del medio impreso.

Los navegadores dominantes evolucionaron desde el pionero Mosaic (1993), liderado por Marc Andreessen y Eric Bina en el NCSA, que fue el primero en mostrar imágenes *inline* junto al texto, hasta Netscape Navigator (1994), que popularizó la web entre el público general. La respuesta de Microsoft fue Internet Explorer (1995), desencadenando la *primera guerra de los navegadores* que duró hasta aproximadamente 2003 con la victoria temporal de IE6 (alcanzó 95 % de cuota de mercado en 2003) y la consiguiente década de estancamiento en estándares web [67].

Web 2.0 — la web social (2004–2014)

El término *Web 2.0* lo popularizó Tim O'Reilly durante una conferencia homónima en octubre de 2004 [43]. Su característica distintiva: la *participación*. Los usuarios pasaron de consumidores a productores de contenido mediante blogs, wikis, redes sociales y plataformas de video. O'Reilly resumió la filosofía como *la web como plataformaz*, donde el valor reside en los datos generados por usuarios y los efectos de red.

Tecnológicamente esta era se sostuvo en cuatro pilares:

- **AJAX** (Asynchronous JavaScript And XML), formalizado por Jesse James Garrett en su artículo seminal de febrero de 2005 [23], que permitió actualizar partes de una página sin recargarla. Google Maps (lanzado ese mismo mes) fue la demostración pública más espectacular del paradigma.
- **Frameworks JavaScript de primera generación**: Prototype (2005), MooTools (2006) y especialmente jQuery (2006), creado por John Resig, que para 2013 estaba presente en más del 60 % de los sitios web del mundo.

- **Plataformas sociales:** Facebook (2004), YouTube (2005), Twitter (2006), GitHub (2008), Instagram (2010), Pinterest (2010).
- **APIs públicas:** Google Maps API (2005), Twitter API (2006), Flickr API, abriendo el camino a los *mashups* y al ecosistema de aplicaciones de terceros.

Esta era también vio surgir las críticas: van Dijck [58] analizó cómo las plataformas sociales construyeron una nueva forma de capitalismo informacional basada en la captura y monetización de datos personales, fenómeno que Zuboff [68] bautizó como *capitalismo de vigilancia*.

Web 3.0 — la web semántica, descentralizada e inteligente (2014–presente)

La tercera era convive con tres significados parcialmente solapados:

1. La *Semantic Web* originalmente propuesta por Berners-Lee, Hendler y Lassila en *Scientific American* en 2001 [8], basada en RDF, OWL y SPARQL, que busca representar el significado de la información para que las máquinas razonen sobre ella. Su realización plena ha sido más lenta de lo previsto, pero subyace en proyectos como *schema.org* (consorcio de Google, Microsoft, Yahoo y Yandex desde 2011), Wikidata y los *Knowledge Graphs* de los buscadores modernos.
2. La *web descentralizada* (Web3 en sentido estricto), emergente desde 2017, que reposa sobre blockchain, IPFS y contratos inteligentes. Wood [63] acuñó el término con la publicación del whitepaper de Ethereum. Sus aplicaciones (dApps) incluyen finanzas descentralizadas (DeFi), tokens no fungibles (NFT) y identidad soberana.
3. La *web inteligente* potenciada por IA generativa, emergente desde 2023 con ChatGPT, Claude y GitHub Copilot. Sus implicaciones en el desarrollo web son tan profundas que algunos autores hablan ya de una *cuarta era* en gestación [31].

En paralelo, las aplicaciones SPA (Single Page Applications) y PWA (Progressive Web Applications) desplazan progresivamente al modelo *server-rendered* clásico, y la integración de IA generativa (2023+) está redefiniendo nuevamente el panorama del desarrollo profesional.

1.2.2 Evolución temporal de los hitos web

La tabla 1.1 sintetiza los hitos cronológicos más relevantes de la trayectoria de la web. No se trata de una lista exhaustiva sino de los eventos que cualquier profesional del desarrollo web debe conocer porque cada uno de ellos transformó la manera de construir software para la web. Los hitos están organizados en cinco grandes movimientos: (a) la fundación (1989–1995), (b) la estandarización (1995–2004), (c) la era social (2004–2014), (d) la madurez técnica (2014–2022) y (e) la era de la inteligencia generativa (2023–presente).

Tabla 1.1: Hitos seleccionados de la historia de las aplicaciones web.

Año	Hito
<i>(a) Fundación</i>	
1989	Tim Berners-Lee propone la WWW en el CERN.
1990	Primer servidor (CERN httpd) y navegador (WorldWideWeb).
1991	Publicación del primer sitio web (info.cern.ch).
1993	Mosaic populariza la navegación con imágenes inline.
1993	Especificación CGI 1.0 para programación del servidor.
1994	Fundación del W3C en MIT; nace Netscape Navigator.
<i>(b) Estandarización</i>	
1995	Brendan Eich crea JavaScript en 10 días para Netscape.
1995	PHP/FI (Personal Home Page Tools) de Rasmus Lerdorf.
1996	CSS1 publicado como recomendación W3C.
1996	Macromedia Flash y los cookies de Netscape.
1998	XML 1.0 estandarizado; nace Google.
1999	ECMAScript 3, base estable de JavaScript por más de una década.
2003	WordPress y MediaWiki marcan la era de los CMS.
<i>(c) Era social</i>	
2004	Web 2.0 (O'Reilly); nace Facebook.
2005	AJAX formalizado; lanzamiento de YouTube; Google Maps.
2006	jQuery; Twitter; Amazon Web Services (S3, EC2).
2008	Google Chrome con motor V8; Android SDK 1.0.
2009	HTML5 borrador funcional WHATWG; Node.js (Ryan Dahl).
2010	Responsive Web Design (Ethan Marcotte).
2011	Laravel (Taylor Otwell); React (Jordan Walke, 2013).
<i>(d) Madurez técnica</i>	
2014	HTML5 oficial W3C; ECMAScript 6 anunciado (publicado en 2015).
2015	ES6 (ES2015); Angular 2 reescrito desde AngularJS.
2018	GDPR; Web Components estables; PWA en producción.
2019	WebAssembly 1.0 estandarizado.
2020	.NET 5 unifica .NET Framework y .NET Core.
2021	OWASP Top 10:2021; HTTP/3 (RFC 9114).
2022	ECMAScript 2022 (ES13); Angular 14 con standalone components.
<i>(e) Era de la IA generativa</i>	
2023	GitHub Copilot, ChatGPT, Claude transforman el desarrollo.
2024	SWEBOK Guide v4.0; PHP 8.3; Spring Boot 3.2; .NET 8 LTS.
2026	ECMAScript 2025; HTML Living Standard al día; IA agentic en IDEs.

1.3 Fundamentos de las aplicaciones web

Los fundamentos de las aplicaciones web no son meros tecnicismos introductorios: constituyen el marco conceptual sobre el que se edifica todo el resto del curso. Sin una comprensión sólida de qué es exactamente una aplicación web, cómo se distingue de otros tipos de software, qué arquitecturas la sostienen y cuáles son sus ventajas y limitaciones, el aprendizaje posterior se vuelve mecánico y superficial. Como observa Shklar y Rosen [55], las aplicaciones web son hoy el género dominante del software: la mayoría de las personas interactúa más horas al día con aplicaciones web (Gmail, redes sociales, banca en línea, sistemas educativos) que con cualquier otro tipo de software.

Esta sección establece las definiciones operativas, la tipología y los modelos arquitectónicos esenciales que el estudiante usará como referencia durante las 18 semanas del curso.

1.3.1 Definición y clasificación de aplicaciones web

Una **aplicación web** es un programa de software cuya lógica se ejecuta en uno o varios servidores remotos y cuya interfaz de usuario se distribuye al cliente a través del Hypertext Transfer Protocol (HTTP) y se renderiza en un navegador o agente de usuario. A diferencia de una aplicación de escritorio, no requiere instalación local; a diferencia de una página web estática, su contenido se genera dinámicamente en respuesta a las acciones del usuario y al estado del servidor [55].

Siguiendo la clasificación de Ganzábal García [14] y la actualización de Nixon [41], las aplicaciones web modernas se agrupan en seis tipologías principales que conviene distinguir:

1. **Aplicaciones web estáticas.** Sirven HTML, CSS y JavaScript pre-compilado sin procesamiento de servidor más allá del envío de archivos. Su contenido cambia solo cuando el desarrollador publica una nueva versión. Ejemplo canónico: un currículum vitae publicado en GitHub Pages, o un blog personal generado con Jekyll o Hugo. **Ventaja:** rendimiento extremo (respuesta en milisegundos desde un CDN). **Limitación:** incapacidad de personalización por usuario.
2. **Aplicaciones web dinámicas.** Generan el HTML en el servidor en cada petición. La lógica reside en lenguajes como PHP, Java, Python, Ruby o C#. Cada usuario ve una versión personalizada del contenido según su sesión, sus permisos, los datos almacenados en bases de datos. **Ejemplo:** el portal de matrícula de la UTEQ; un panel de administración construido con Laravel; cualquier sitio WordPress.
3. **Aplicaciones de página única (SPA).** Cargan un único HTML inicial y a partir de ahí toda la interacción ocurre vía JavaScript que consume APIs REST o GraphQL. La navegación entre páginas ocurre sin recargar el navegador, gracias a la History API. **Ejemplos:** Gmail, Trello, Notion. Frameworks habituales en 2026: React 19, Angular 19, Vue 3.5, Svelte 5.
4. **Aplicaciones web progresivas (PWA).** Combinan SPA con capacidades nativas: instalación en pantalla de inicio, notificaciones push, funcionamiento offline, acceso a sensores del dispositivo. **Tecnologías:** Service Workers, Web App Manifest, IndexedDB. Twitter Lite fue uno de los primeros casos de éxito; en Latinoamérica, Pinterest PWA y MercadoLibre han demostrado mejoras del 60 % en retención de usuarios [51].
5. **Aplicaciones de comercio electrónico.** Subgénero especializado con flujos transaccionales: catálogo, carrito, pasarela de pago (PayPal, Stripe, Datafast en Ecuador), gestión de pedidos, post-venta. Plataformas dominantes: Shopify, WooCommerce, Magento/Adobe Commerce.
6. **Portales corporativos y e-government.** Aplicaciones con altos requisitos de seguridad, trazabilidad e integración con sistemas heredados. Ejemplos nacionales: el portal del SRI, el sistema de compras públicas SOCE, la Ventanilla Única Electrónica del SENA. Suelen requerir autenticación con certificado digital y firma electrónica.

1.3.2 Características de las aplicaciones web modernas

Antes de profundizar en arquitectura y ventajas comparativas, conviene sintetizar las características esenciales que distinguen a una aplicación web de cualquier otro tipo de software. Estas características no son meras curiosidades técnicas: cada una tiene implicaciones directas en cómo se desarrollan, despliegan, usan, escalan y aseguran las aplicaciones web. Newman [40] y Kleppmann [30] dedican capítulos enteros a discutir las consecuencias arquitectónicas de cada característica.

Tabla 1.2: Características esenciales de las aplicaciones web modernas y sus implicaciones.

Dimensión	Característica
<i>Desarrollo</i>	
Lenguajes	Múltiples coexistentes: HTML+CSS+JS en cliente; PHP/Java/C#/Node en servidor.
Herramientas	IDE (IntelliJ IDEA Ultimate, VS Code), Git, Docker, gestores de paquetes.
Metodología	Predominantemente ágil (Scrum, Kanban) con CI/CD.
<i>Despliegue</i>	
Empaquetado	Contenedores Docker, archivos JAR/WAR, paquetes <code>npm</code> .
Hosting	Cloud (AWS, Azure, GCP), VPS, on-premise; orquestación con Kubernetes.
Actualización	<i>Rolling updates</i> sin interrumpir el servicio; <i>blue-green deployment</i> .
<i>Uso</i>	
Acceso	Universal vía URL, sin instalación local.
Multiplataforma	Mismo código accesible desde Windows, macOS, Linux, iOS, Android.
Conectividad	Requiere red; PWAs ofrecen modo offline limitado.
<i>Operación</i>	
Monitoreo	Métricas (Prometheus), logs (ELK), traces (OpenTelemetry, Tempo).
Escalabilidad	Horizontal preferida sobre vertical en arquitecturas modernas.
Disponibilidad	Objetivo común 99.9% (8.7 h/año de downtime aceptable).
<i>Seguridad</i>	
Superficie de ataque	Pública en internet: requiere defensas multinivel.
Estándares	OWASP Top 10:2021, ISO 27001, GDPR, LOPDP (Ecuador).
Identidad	OAuth 2.0, OpenID Connect, SAML, JWT.

Las implicaciones prácticas de estas características son enormes. Por ejemplo, la naturaleza multiplataforma elimina el costo histórico de mantener versiones separadas para Windows, macOS y Linux, pero introduce el reto de la consistencia entre navegadores. La actualización *rolling* elimina la fricción del ríndia de instalaciónz del software clásico, pero exige diseñar el código para que dos versiones coexistan durante el despliegue. La superficie pública de ataque obliga a integrar la seguridad desde el día uno; no se puede áagregar seguridadz al final como en un sistema interno.

1.3.3 Arquitectura cliente-servidor en aplicaciones web

La arquitectura cliente-servidor es el modelo de comunicación distribuida que organiza prácticamente la totalidad del tráfico web actual. Comprenderla a profundidad es indispensable porque cada decisión técnica posterior (desde qué lenguaje usar en el navegador hasta cómo escalar el servidor) se apoya en sus principios. En esta sección revisaremos primero su concepto fundamental, después la diferencia frecuentemente confundida entre *capas* y *niveles*, y finalmente las arquitecturas de 2, 3 y N niveles con ejemplos concretos y diagramas.

Concepto fundamental

La arquitectura cliente-servidor es el paradigma fundamental sobre el que se erige la World Wide Web. En este modelo, un proceso *cliente* emite una solicitud (*request*) a un proceso *servidor*, el cual procesa la solicitud y devuelve una respuesta (*response*). El protocolo que estandariza este intercambio es HTTP/1.1 (RFC 7230–7235), HTTP/2 (RFC 7540) o HTTP/3 (RFC 9114), este último ya dominante en 2026 gracias a su desempeño superior en redes móviles.

Anatomía de una petición HTTP

```
GET /productos?categoria=cafe HTTP/1.1
Host: tienda.uteq.edu.ec
Accept: text/html
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Accept-Language: es-EC,es;q=0.9,en;q=0.8
Cookie: SESSID=ab39e1ff7e1c4
```

Listing 1.1: Ejemplo de Shell

La primera línea es la *línea de petición*, compuesta por método, ruta y versión del protocolo. Las líneas siguientes son *cabeceras*; tras una línea en blanco puede ir el cuerpo (en métodos POST, PUT y PATCH).

Niveles vs capas: una distinción frecuentemente confundida

Uno de los conceptos más confusos para el estudiante principiante es la distinción entre *capas* y *niveles*. Aunque a menudo se usan como sinónimos, en arquitectura de software profesional **tienen significados diferentes** que conviene precisar desde el inicio [20, 51]:

Capa (*layer*) vs Nivel (*tier*)

Capa (*layer*): agrupación *lógica* de responsabilidades dentro del software. Es una abstracción conceptual; varias capas pueden ejecutarse en el mismo proceso físico. Ejemplo: en una aplicación monolítica, las capas *Presentación*, *Negocio* y *Datos* pueden coexistir en el mismo `.jar`.

Nivel (*tier*): unidad *física* de despliegue. Cada nivel corre típicamente en su propio servidor o contenedor, y los niveles se comunican por red. Ejemplo: *servidor de aplicación* y *servidor de BD* son dos niveles distintos aunque la app sea un monolito de varias capas internas.

Un sistema puede tener **3 capas pero 2 niveles** (capas presentación, negocio y datos en un mismo proceso que se conecta a una BD en otro servidor), o **3 capas y 3 niveles** (cada capa en su propio servidor).

Arquitecturas N-niveles: panorama visual

Las arquitecturas web se clasifican por la cantidad de *niveles físicos* (máquinas o procesos separados) en que se despliega el sistema. Comprender visualmente esta diferencia entre 1, 2, 3 y

Niveles es esencial antes de hablar de patrones más complejos como microservicios o serverless. La siguiente figura contrasta los cuatro casos canónicos.

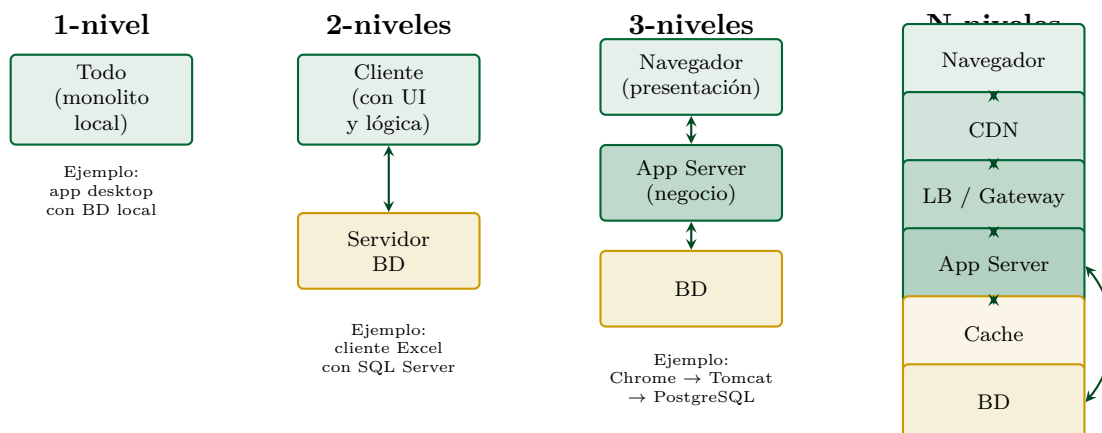


Figura 1.1: Las cuatro variantes principales de arquitectura cliente-servidor por número de niveles físicos. La elección depende del tamaño, presupuesto, requisitos de escalabilidad y disponibilidad. La mayoría de aplicaciones web modernas usan 3 o más niveles.

A continuación se describe cada variante:

Arquitectura 1-nivel (monolito local). Todo el software (UI, lógica, datos) reside en una sola máquina sin red. No aplica a la web. Ejemplos: una aplicación de escritorio con SQLite embebido, Microsoft Access en modo monousuario.

Arquitectura 2-niveles (cliente-servidor tradicional). El cliente contiene la UI y parte de la lógica de negocio; el servidor expone solo la BD. Modelo dominante en los 90 con clientes *fat-client* (Visual Basic + Oracle, Delphi + Sybase). Para la web no es óptimo: la lógica de negocio expuesta al cliente compromete la seguridad. Ejemplos vigentes: algunas aplicaciones legadas de contabilidad con cliente Windows + SQL Server.

Arquitectura 3-niveles (la clásica web). Separa explícitamente presentación (navegador), lógica de negocio (servidor de aplicación: Tomcat, IIS, Apache+PHP) y datos (SGBD). Es el modelo canónico de la web desde 1998 y sigue siendo el más común. Ejemplo: Chrome → Spring Boot → PostgreSQL.

Arquitectura N-niveles. Introduce niveles adicionales para abordar requisitos no funcionales: CDN para contenido estático, balanceador de carga para distribución, gateway API para autenticación cruzada, caché distribuida (Redis), sistemas de mensajería (Kafka). Es el modelo dominante en sistemas a gran escala [3, 30]. Ejemplo: Netflix opera con decenas de niveles distintos.

Anatomía detallada del flujo de una petición web

Una aplicación web real involucra al menos cinco actores físicos o lógicos cuya comprensión es esencial para diagnosticar problemas en producción:

1. **Navegador del usuario:** Chrome, Firefox, Safari, Edge. Procesa HTML, CSS y JavaScript;

gestiona cookies, caché y almacenamiento local.

2. **Servidor DNS:** traduce el nombre de dominio (`uteq.edu.ec`) a una dirección IP. Operación que dura típicamente entre 20–80 ms.
3. **Balanceador o proxy inverso:** Nginx, HAProxy, AWS ELB. Distribuye la carga entre servidores de aplicación, termina TLS, sirve contenido estático.
4. **Servidor de aplicación:** Apache HTTPD, Tomcat, IIS, Node.js, PHP-FPM. Ejecuta el código de la aplicación.
5. **Sistema gestor de bases de datos:** PostgreSQL, MySQL, MariaDB, SQL Server, Oracle. Persiste el estado de la aplicación.

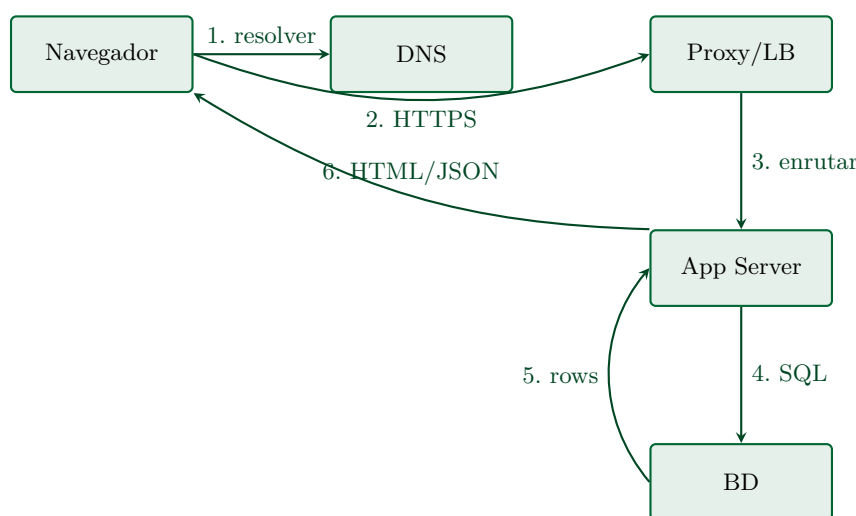


Figura 1.2: Flujo simplificado de una petición web. Cada uno de estos seis pasos puede convertirse en cuello de botella; la disciplina de optimización web consiste en medir cada eslabón.

1.3.4 Ventajas y desventajas de las aplicaciones web

Toda decisión arquitectónica implica trade-offs: las aplicaciones web ofrecen ventajas operativas significativas frente a las nativas, pero también limitaciones que debemos conocer antes de comprometernos con la plataforma. La tabla siguiente resume el balance.

Tabla 1.3: Comparativa entre aplicaciones web, escritorio y móviles nativas.

Criterio	Web	Escritorio	Móvil nativa
Despliegue	Inmediato sin instalación	Empaquetado e instalador	Tienda de apps (revisión)
Compatibilidad	Multiplataforma	Sistema-dependiente	iOS y Android (separadas)
Acceso a hardware	Limitado vía Web APIs	Total	Total
Costo de desarrollo	Bajo (un <i>stack</i>)	Medio	Alto (dos <i>stacks</i>)
Costo de mantenimiento	Muy bajo (deploy único)	Medio	Alto (dos versiones)
Rendimiento bruto	Medio	Alto	Alto
Modo <i>offline</i>	Limitado (PWA)	Total	Total
Distribución	Instantánea (URL)	Descarga manual	Tienda de apps
Actualizaciones	Automáticas	Manuales o semi	Aceptación del usuario

1.4 Diseño web inclusivo y accesibilidad

La web nació con vocación universal: cualquier persona, desde cualquier dispositivo, en cualquier lugar del mundo, debería poder acceder a la información que contiene. Sin embargo, la realidad muestra que millones de personas con discapacidad encuentran barreras infranqueables cada día al navegar sitios que ignoraron sus necesidades. La accesibilidad web no es un tema marginal ni un añadido decorativo: es una responsabilidad profesional, una obligación legal en cada vez más países (incluido Ecuador) y un factor de competitividad comercial cuantificable. Esta sección introduce el concepto, presenta las cuatro categorías de discapacidad que afectan la experiencia web, expone las pautas WCAG 2.1 del W3C con sus principios POUR, y termina con los criterios de éxito que todo desarrollador debe dominar.

1.4.1 ¿Qué es la accesibilidad web?

La accesibilidad web es la práctica de diseñar y construir sitios y aplicaciones que puedan ser percibidos, comprendidos, navegados y utilizados por *todas las personas*, independientemente de sus capacidades físicas, sensoriales o cognitivas. La definición canónica del W3C [66] la formula así: *la accesibilidad significa que personas con discapacidades pueden percibir, comprender, navegar e interactuar con la Web, y que pueden contribuir a la Web*.

Sin embargo, la accesibilidad web no es solo una buena causa moral: es un imperativo legal, comercial y técnico. García-Holgado et al. [24] demuestran en un estudio sistemático sobre instituciones de educación superior que solo el 23 % de los sitios analizados cumplen el nivel mínimo de WCAG 2.1, lo que constituye un incumplimiento normativo en la mayoría de jurisdicciones.

Marco legal de la accesibilidad

La accesibilidad no es solo una buena práctica: en muchas jurisdicciones es una obligación legal con sanciones concretas. La siguiente enumeración recopila los marcos normativos vigentes que un desarrollador web debe conocer antes de publicar un sitio.

- **Ecuador.** La Ley Orgánica de Discapacidades (Registro Oficial 2012) exige conformidad mínima AA en sitios públicos a partir de su Reglamento de 2017. CONADIS supervisa el cumplimiento.
- **Estados Unidos.** Section 508 del Rehabilitation Act (enmienda de 1998, refundición de 2017) obliga a las agencias federales a la accesibilidad. Casos como *Robles v. Domino's Pizza* (2019) extendieron la jurisprudencia al sector privado bajo el ADA.
- **Unión Europea.** Directiva 2016/2102 sobre accesibilidad de los sitios y aplicaciones móviles de los organismos del sector público. EN 301 549 V3.2.1 (2021) define los requisitos técnicos.
- **España, México, Argentina, Brasil, Colombia, Chile.** Todos cuentan con normativa específica derivada de los lineamientos internacionales.

Impacto comercial cuantificable

Más allá del cumplimiento legal, la accesibilidad genera retorno medible:

- **15 % de la población mundial** vive con alguna forma de discapacidad según la OMS (2024).
- Mejorar la accesibilidad mejora el SEO: Google penaliza sitios con baja accesibilidad mediante Core Web Vitals.
- Microsoft estimó que sus inversiones en accesibilidad generaron un retorno de inversión del 1.000 % al ampliar mercados [37].

1.4.2 Las cuatro categorías de discapacidad relevantes para la web

Antes de profundizar en los criterios técnicos, es indispensable comprender qué tipos de barreras enfrentan los usuarios con discapacidad y qué tecnologías de asistencia emplean. Esta comprensión es la base ética y empática de la accesibilidad: no se trata de cumplir una norma abstracta sino de habilitar a personas reales con necesidades específicas.

Las categorías que se presentan a continuación siguen la taxonomía clásica de Henry [24] y el W3C WAI. Importante: una misma persona puede tener *múltiples* discapacidades simultáneas (por ejemplo, baja visión y artritis), o *discapacidades temporales* (un brazo enyesado, una lesión ocular) o *situacionales* (usar el móvil bajo luz solar intensa, manejar con manos ocupadas). El diseño accesible beneficia a todos estos casos.

1. **Discapacidad visual.** Incluye ceguera total (1.5 % de la población mundial), baja visión y daltonismo (8 % de hombres, 0.5 % de mujeres). Los usuarios con ceguera utilizan lectores de pantalla (NVDA gratuito en Windows, JAWS comercial, VoiceOver en Apple, TalkBack en Android) que recorren el árbol de accesibilidad del DOM y convierten la información a voz. Los daltónicos requieren que la información no se transmita exclusivamente por color (el rojo y verde son indistinguibles para la mayoría).
2. **Discapacidad auditiva.** Sordera total o parcial (5 % de la población mundial). Requiere subtítulos en contenido multimedia (WCAG 1.2.2), transcripciones de podcasts, lengua de señas intérprete en videos institucionales críticos, y notificaciones visuales además de sonoras.
3. **Discapacidad motora.** Imposibilidad de usar mouse o teclado de forma estándar. Causas incluyen parálisis, temblores, artritis reumatoide, distrofia muscular, amputaciones. Requiere navegación completa por teclado (sin depender del mouse), áreas de clic amplias (mínimo 44×44 px según WCAG 2.5.5), ausencia de tiempos límite estrictos, soporte para entrada por voz, switches adaptados.
4. **Discapacidad cognitiva.** La categoría más amplia y diversa: incluye dislexia (10 % de la población), TDAH (5 %), trastornos del espectro autista (1–2 %), discapacidad intelectual, demencia. Requiere lenguaje claro y conciso, estructura predecible, ausencia de elementos

parpadeantes (riesgo de crisis epilépticas fotosensibles entre 3–65 Hz), feedback claro de errores, opciones para ajustar tipografía y espaciado.

1.4.3 Las Web Content Accessibility Guidelines 2.1 (WCAG 2.1)

WCAG 2.1 [66] es la norma internacional de referencia, publicada por el W3C en junio de 2018 y vigente. Está organizada en torno a cuatro principios fundamentales conocidos por el acrónimo **POUR** (Perceivable, Operable, Understandable, Robust). Cada principio define directrices, cada directriz se materializa en *criterios de éxito* comprobables, y cada criterio se clasifica en uno de tres niveles de conformidad: A (mínimo), AA (recomendado), AAA (excepcional).

Esta sección no se limita a enunciar los principios: para cada uno explica *qué tecnologías* permiten implementarlo y *cómo* medir el cumplimiento de forma objetiva.

Principio 1 — Perceptible

Enunciado: La información y los componentes de la interfaz deben presentarse de modo que los usuarios puedan percibirlos.

Cómo lograrlo en HTML:

- Atributo `alt` descriptivo en toda `<img>`.
- Subtítulos `<track kind="captions">` en `<video>`.
- Transcripciones para podcasts publicadas junto al audio.
- `aria-label` para iconos clicables sin texto.

Cómo lograrlo en CSS:

- Contraste mínimo 4.5:1 (texto normal) y 3:1 (texto grande).
- No usar solo color para transmitir información crítica.
- Diseño responsive que soporte zoom hasta 200 % sin pérdida funcional.

Cómo medir cumplimiento:

- **WebAIM Contrast Checker** (<https://webaim.org/resources/contrastchecker/>): ingresa colores hex y devuelve la ratio exacta.
- **axe DevTools** (extensión Chrome/Firefox): detecta imágenes sin `alt`, contraste insuficiente, audio sin transcripción.
- **Pa11y CLI** (<https://pa11y.org>): herramienta de línea de comandos para integrar auditoría en CI/CD.
- **Lighthouse Accessibility** (incluido en Chrome DevTools): score 0–100 con recomendaciones priorizadas.

Principio 2 — Operable

Enunciado: Los componentes y la navegación deben ser operables por todos los usuarios.

Cómo lograrlo en HTML:

- Elementos interactivos nativos (`<button>`, `<a>`) en lugar de `<div onclick>`.
- Atributo `tabindex` solo cuando estrictamente necesario; nunca `tabindex="2"` u otros valores positivos.
- `skip-link` al inicio de cada página para saltar a contenido principal.

Cómo lograrlo en CSS:

- `:focus-visible` con outline claramente visible.
- `prefers-reduced-motion` respetado en animaciones.
- Áreas de clic mínimo 44×44 px.

Cómo lograrlo en JavaScript:

- Sin tiempo límite ajustable o que pueda extenderse.
- Manejo de teclado en componentes custom: **Enter**, **Space**, **Escape**, flechas direccionales.
- Sin contenido parpadeante en rango 3–55 Hz.

Cómo medir cumplimiento:

- Navegación completa solo con **Tab** y **Enter** sin mouse.
- Recorrido visual del foco siguiendo orden lógico.
- Lighthouse y axe DevTools detectan tabbing roto.

Principio 3 — Comprensible

Enunciado: La información y el manejo de la interfaz deben ser comprensibles.

Cómo lograrlo en HTML:

- Atributo `lang="es-EC"` en `<html>`; cambiar con `<span lang="en">` en fragmentos en otros idiomas.
- `<label for="id">` asociado a cada `<input>`.
- Atributo `autocomplete` con valores estándares.
- `aria-describedby` para descripción extendida.

Cómo lograrlo en lenguaje natural:

- Nivel de lectura grado 8–9 según escala Flesch-Kincaid.
- Frases cortas (máx. 25 palabras).
- Voz activa preferida a pasiva.
- Glosario de términos técnicos accesible desde cada uso.

Cómo medir cumplimiento:

- **Hemingway Editor** (<https://hemingwayapp.com>): evalúa legibilidad.
- Pruebas de usabilidad con usuarios reales de baja alfabetización digital.

Principio 4 — Robusto

Enunciado: El contenido debe ser robusto suficiente para que pueda ser interpretado por una amplia variedad de agentes de usuario, incluyendo tecnologías asistivas.

Cómo lograrlo en HTML:

- HTML válido sin errores (verificar en <https://validator.w3.org/nu/>).
- Roles ARIA solo cuando los elementos semánticos nativos no basten.
- IDs únicos en todo el documento.
- Nombres accesibles para todos los controles (`aria-label` o texto visible asociado).

Cómo medir cumplimiento:

- W3C Markup Validator: 0 errores.
- NVDA o VoiceOver: recorrido auditivo manual.
- axe DevTools: 0 violaciones serias o críticas.

Niveles de conformidad y herramientas integradas de auditoría

WCAG 2.1 establece tres niveles de conformidad: **A** (mínimo imprescindible), **AA** (recomendado para sector público y privado serio) y **AAA** (excepcional, raro de alcanzar íntegramente). En 2023 se publicó WCAG 2.2 que añadió nueve criterios de éxito adicionales y previsiblemente WCAG 3.0 (*Silver*, borrador desde 2021) será la próxima revolución mayor, reemplazando el sistema de niveles A/AA/AAA por un puntaje continuo.

1.4.4 Criterios de éxito clave de WCAG 2.1

Esta subsección presenta los diez criterios de éxito más importantes que todo desarrollador web debe interiorizar. Para cada criterio se explica: *(a)* qué exige exactamente la norma, *(b)* cómo

implementarlo en código real, y (c) un ejemplo concreto.

Diez criterios WCAG imprescindibles — detallados

1.1.1 Contenido no textual (Nivel A).

Qué exige: todo contenido no textual tiene una alternativa textual equivalente.

Cómo lograrlo: `alt` en `<img>`; `aria-label` en iconos clicables; `<title>` en SVGs significativas; `alt=""` (vacío) si la imagen es decorativa.

Ejemplo: ``.

1.3.1 Información y relaciones (Nivel A).

Qué exige: la estructura semántica del HTML refleja la jerarquía visual y las relaciones de la información.

Cómo lograrlo: usar encabezados correctos `<h1>`-`<h6>`, listas `<ul>`/`<ol>` para listas, `<table>` con `<th scope>` para datos tabulares, `<fieldset>`+`<legend>` para agrupar campos relacionados.

Anti-ejemplo: simular tabla con `<div>` y CSS Grid sin roles ARIA, rompe lectores de pantalla.

1.4.3 Contraste mínimo (Nivel AA).

Qué exige: ratio de contraste $\geq 4.5:1$ para texto normal; $\geq 3:1$ para texto grande (18pt+ o 14pt+ negrita).

Cómo lograrlo: verificar con WebAIM Contrast Checker; usar paletas pre-validadas.

Ejemplo de combinaciones AA conformes sobre fondo blanco: #006633 (verde UTEQ) ratio 6.27:1; negro #000000 ratio 21:1.

1.4.10 Reflujo (Nivel AA).

Qué exige: el contenido reflowea sin pérdida de información ni funcionalidad y sin scroll horizontal en 320 px de ancho.

Cómo lograrlo: CSS responsive mobile-first; `viewport meta`; unidades relativas (rem, em, %).

1.4.11 Contraste no textual (Nivel AA).

Qué exige: componentes UI (botones, iconos de estado, bordes de campos) con contraste $\geq 3:1$ con sus alrededores.

Cómo lograrlo: bordes visibles en inputs; iconos de estado con fondo o borde definido.

2.1.1 Teclado (Nivel A).

Qué exige: toda funcionalidad disponible solo desde teclado.

Cómo lograrlo: usar elementos interactivos nativos; manejar **Enter** y **Space** en componentes custom; no atrapar el foco.

Anti-ejemplo: dropdown que solo abre con hover de mouse.

2.4.7 Foco visible (Nivel AA).

Qué exige: indicador visible cuando un elemento recibe foco.

Cómo lograrlo: no quitar el outline por defecto, o reemplazarlo con `:focus-visible` de mayor visibilidad (≥ 2 px, contraste $\geq 3:1$).

3.1.1 Idioma de la página (Nivel A).

Qué exige: elemento `<html>` tiene atributo `lang` correcto.

Cómo lograrlo: `<html lang="es-EC">`.

3.3.1 Identificación de errores (Nivel A).

Qué exige: si el sistema detecta error en entrada del usuario, lo identifica y describe en texto.

Cómo lograrlo: `<input aria-invalid="true"aria-describedby="err-email">` y mensaje en `<span id="err-email">Email inválido</span>`.

4.1.2 Nombre, función, valor (Nivel A).

Qué exige: para todos los componentes de interfaz, nombre, función y valor son determinables programáticamente.

Cómo lograrlo: usar elementos nativos siempre que sea posible; si se construye un componente custom, declarar `role`, `aria-label`, `aria-checked` etc.

1.5 Configuración del entorno con IntelliJ IDEA Ultimate

Antes de continuar con el resto de la asignatura, el estudiante debe disponer de un entorno IDE funcional. El siguiente ejercicio resuelto guía paso a paso la instalación y verificación de IntelliJ IDEA Ultimate con la licencia educativa UTEQ.

Ejercicio 1.1 (RESUELTO) — Instalación y verificación de IntelliJ IDEA Ultimate

Objetivo: configurar el entorno integrado de desarrollo que se utilizará durante todo el semestre y verificar que los plugins necesarios para HTML, CSS y JavaScript están operativos.

Paso 1 — Obtener la licencia educativa. Ingresar a <https://www.jetbrains.com/community/education> y crear una cuenta con el correo institucional `<usuario>@uteq.edu.ec`. Al verificar el dominio, JetBrains envía un correo de confirmación con una clave educativa renovable anualmente. Sin esta clave, IntelliJ IDEA Ultimate funciona únicamente 30 días en modo evaluación.

Paso 2 — Descargar e instalar. Descargar IntelliJ IDEA Ultimate desde <https://www.jetbrains.com/idea/download>. El instalador pesa aproximadamente 850 MB.

Paso 3 — Activar la licencia. En la primera ejecución elegir `¡Actívala! License` → `JB Accountz`. Verificar en `Help` → `Register` que la licencia es de tipo *Educational License*.

Paso 4 — Crear el primer proyecto. En la pantalla *Welcome* pulsar `New Project`. Seleccionar la plantilla `HTML`, establecer **Name:** `appweb-semana01`, y pulsar `Create`.

Paso 5 — Verificar plugins esenciales en `File` → `Settings` → `Plugins`: `HTML Tools`, `CSS`, `JavaScript and TypeScript`, `Markdown`, `Live Edit`.

Paso 6 — Probar. Abrir el archivo `index.html` que el IDE generó automáticamente. Posicionar el cursor sobre el código y pulsar `Alt+F2`; elegir `Chrome`.

Resultado esperado (verificación):

[Ventana del navegador Chrome]
 URL: http://localhost:63342/appweb-semana01/index.html
 [Pestaña: index.html]
 Hello, World!
(Página HTML mínima generada por IntelliJ con el texto por defecto, servida desde el servidor de desarrollo embebido del IDE en el puerto 63342.)

Indicadores de éxito verificables:

- La barra de URL muestra una dirección localhost:63342/...
- El título de la pestaña coincide con index.html.
- En IntelliJ, el panel *Run* (parte inferior) muestra el servidor en estado *Running*.

Ejercicio 1.2 (RESUELTO) — Primera página HTML5 semántica y accesible

Objetivo: crear una página HTML5 estructuralmente correcta, semánticamente significativa y accesible. Verificar su validez en el W3C Validator y la ausencia de violaciones críticas en axe DevTools.

Código completo (Listado 1.2):

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7   <title>Mi primera pagina semantica - UTEQ</title>
8   <meta name="description" content="Pagina de practica
9     para Aplicaciones Web - UTEQ 2026">
10 </head>
11 <body>
12   <header role="banner">
13     <h1>Universidad Tecnica Estatal de Quevedo</h1>
14     <p>La primera universidad agropecuaria del Ecuador</p>
15     <nav role="navigation" aria-label="Menu principal">
16       <ul>
17         <li><a href="#inicio">Inicio</a></li>
18         <li><a href="#carreras">Carreras</a></li>
19         <li><a href="#contacto">Contacto</a></li>
20       </ul>
21     </nav>
22   </header>
23   <main role="main">
24     <article>
25       <h2 id="inicio">Bienvenido al curso de Aplicaciones Web</h2>
26       <p>Este es el primer ejercicio del semestre. La fecha de hoy es
27         <time datetime="2026-05-04">4 de mayo de 2026</time>.</p>
28       <figure>
29         

```

```

31     <figcaption>Escudo institucional UTEQ</figcaption>
32   </figure>
33 </article>
34 <aside aria-label="Informacion adicional">
35   <h2>Datos de contacto</h2>
36   <address>
37     Av. Quito km. 1.5 via a Santo Domingo<br>
38     Quevedo, Los Rios, Ecuador<br>
39     Telefono: <a href="tel:+59353702220">(+593) 5 3702-220</a>
40   </address>
41 </aside>
42 </main>
43 <footer role="contentinfo">
44   <small>&copy; 2026 UTEQ. Todos los derechos reservados.</small>
45 </footer>
46 </body>
47 </html>

```

Listing 1.2: index.html (Ejercicio 1.2)

Explicación condensada: `<!DOCTYPE html>` declara HTML5; `lang="es-EC"` permite al lector de pantalla elegir voz española; `<meta charset>` evita corrupción de tildes; `<meta viewport>` habilita responsive en móvil. Los roles ARIA (`banner`, `navigation`, `main`, `contentinfo`) refuerzan la semántica para tecnologías asistivas antiguas. `<time datetime>` hace la fecha legible por máquinas. `alt` en la imagen cumple WCAG 1.1.1.

Resultado esperado en el navegador:

[Vista renderizada en Chrome - sin estilos CSS]

Universidad Tecnica Estatal de Quevedo

La primera universidad agropecuaria del Ecuador

• Inicio • Carreras • Contacto

Bienvenido al curso de Aplicaciones Web

Este es el primer ejercicio del semestre. La fecha de hoy es 4 de mayo de 2026.



Escudo institucional UTEQ

Datos de contacto

Av. Quito km. 1.5 via a Santo Domingo

Quevedo, Los Rios, Ecuador

Telefono: (+593) 5 3702-220

© 2026 UTEQ. Todos los derechos reservados.

Verificación obligatoria del éxito:

1. W3C Validator (<https://validator.w3.org/nu/>): copiar el código completo, pegar

en `Validate by direct input`z, pulsar Check. Debe aparecer en verde: `Document checking completed. No errors or warnings to show.`z

2. *axe DevTools*: instalar la extensión desde Chrome Web Store. F12 → pestaña axe DevTools → `Scan ALL of my page`z. Resultado esperado: 0 violaciones críticas.
3. *Navegación con teclado*: pulsar Tab repetidamente desde la barra de URL. El foco debe recorrer en orden los tres enlaces del menú y el enlace del teléfono.

Ejercicio 1.3 (PROPUESTO) — Página de presentación personal accesible

Objetivo: construir una página de presentación personal que integre todos los conceptos vistos: estructura semántica, accesibilidad, metadatos correctos, navegación con teclado.

Especificaciones obligatorias:

1. Crear un nuevo proyecto en IntelliJ: `appweb-presentacion-<TuApellido>`.
2. El archivo `index.html` debe contener:
 - DOCTYPE HTML5 y `lang="es-EC"`.
 - Metadatos: `charset`, `viewport`, `description` (máx. 160 caracteres), `author`, `keywords` (5–7 términos).
 - `<header>` con nombre completo (`h1`), foto con `alt` descriptivo, y un `<nav>` con cinco enlaces internos.
 - Cinco `<section>` (cada una con su `<h2>`): `¿Sobre mí?`, `¿Estudios?`, `¿Habilidades técnicas?`, `¿Proyectos?` y `¿Contacto?`.
 - En `¿Habilidades técnicas?`, una tabla `<table>` con cuatro filas (HTML, CSS, JavaScript, otra de elección) y tres columnas (tecnología, nivel 1–5, años de experiencia).
 - En `¿Proyectos?`, tres `<article>` con `<figure>+<figcaption>` cada uno.
 - `<footer>` con `copyright` y fecha `<time datetime="2026">`.
3. La página debe ser *válida* en <https://validator.w3.org/nu/> sin errores ni advertencias.
4. axe DevTools debe arrojar cero violaciones de cualquier severidad.
5. La navegación por teclado debe permitir alcanzar todos los enlaces y secciones.

Entrega: `index.html` + captura del W3C Validator mostrando `¿No errors?` + captura de axe DevTools mostrando `¿No issues found?`.

Esta práctica forma parte de la *Sumativa GA-Taller: Encuadre académico*.

1.6 Buenas prácticas profesionales para HTML semántico

Las buenas prácticas profesionales no son convenciones arbitrarias: cada una tiene una justificación técnica, de mantenibilidad, de accesibilidad o de SEO documentada en la literatura especializada [53, 15]. La adherencia disciplinada a estas prácticas es lo que separa al código profesional del código amateur.

Doce reglas de oro del HTML semántico profesional

1. **Un solo <h1> por página**, que coincide con el título principal y suele ser similar al del <title>.
2. **Jerarquía de encabezados sin saltos**: $h1 \rightarrow h2 \rightarrow h3$, nunca $h1 \rightarrow h3$ saltando $h2$.
3. **Ningún encabezado vacío y ningún encabezado meramente decorativo**: si un texto necesita estilo de título pero no es título, usar <p> con clase CSS.
4. **Listas para listas**: cualquier serie de elementos similares debe ir en , o <dl>, nunca en una serie de <div>.
5. **Tablas solo para datos tabulares**, nunca para maquetación.
6. **Botones para acciones, enlaces para navegación**.
7. **Imágenes con alt descriptivo**; si la imagen es puramente decorativa, alt="" (vacío).
8. **Formularios con <label> asociado a cada <input>**.
9. **Validar siempre** con <https://validator.w3.org/nu/> antes de cada commit.
10. **Atributo lang obligatorio** en <html>.
11. **Evitar
 para crear espacios**: usar CSS.
12. **Mantener IDs únicos**: dos elementos con el mismo id son HTML inválido y rompen tecnologías asistivas.

1.7 Tabla de referencia rápida: tags HTML5 más usados, atributos y valores

La tabla 1.4 compendia los tags HTML5 más frecuentes en el desarrollo profesional, sus atributos principales, los valores admitidos y el efecto esperado. Algunos atributos (id, class, lang, title, hidden, tabindex, eventos onclick etc.) son **globales**: aplican a casi todos los elementos. Estos se listan una sola vez al inicio para evitar repetición.

Tabla 1.4: Tags HTML5 más usados: estructura, contenido y formularios.

Tag	Atributo	Valores / efecto	Ejemplo
<i>Estructura semántica (HTML5)</i>			
<header>	—	Cabecera de página o sección	<header>...</header>
<nav>	—	Bloque de navegación	<nav>...</nav>
<main>	—	Contenido principal (único por página)	<main>...</main>
<article>	—	Contenido autónomo reutilizable	<article>...</article>
<section>	—	Sección temática con encabezado	<section>...</section>
<aside>	—	Contenido relacionado pero secundario	<aside>...</aside>
<footer>	—	Pie de página o sección	<footer>...</footer>
<i>Contenido textual</i>			
<h1>–<h6>	—	Encabezados de niveles 1 a 6	Titulo
<p>	—	Párrafo	Texto.
	—	Énfasis fuerte (importancia)	Atencion
	—	Énfasis (entonación)	realmente
<mark>	—	Texto destacado contextualmente	<mark>resaltado</mark>

Continúa en la siguiente página. . .

(Continuación de la Tabla 1.4)

Tag	Atributo	Valores / efecto	Ejemplo
<time>	datetime	Fecha/hora ISO 8601 legible máquina	<time datetime="2026-05-04">
<code>	—	Fragmento de código	<code>let x=1</code>
<blockquote>	cite	URL fuente de la cita	<blockquote cite="...">
<i>Hipervínculos y enlaces</i>			
<a>	href	URL destino	href="https://uteq.edu.ec"
	target	_blank, _self, _parent	target="_blank"
	rel	noopener, noreferrer, me	rel="noopener"
	download	Fuerza descarga	download="archivo.pdf"
<i>Imágenes y multimedia</i>			
	src	URL de la imagen	src="logo.png"
	alt	Texto alternativo (obligatorio)	alt="Logo UTEQ"
	width, height	Dimensiones (px o %)	width="200"
	loading	lazy, eager	loading="lazy"
	srcset	Múltiples resoluciones	srcset="img-2x.png 2x"
<video>	src	URL del video	src="video.mp4"
	controls	Booleano: controles nativos	controls
	poster	Imagen previa	poster="prev.jpg"
	preload	none, metadata, auto	preload="metadata"
<i>Listas</i>			
,	—	Lista no/ordenada	...
	—	Ítem de lista	texto
<dl>, <dt>, <dd>	—	Lista de definiciones	—
<i>Tablas</i>			
<table>	—	Datos tabulares	<table>...</table>
<thead>, <tbody>, <tfoot>	—	Secciones	—
<tr>	—	Fila	<tr>...</tr>
<th>	scope	col, row, colgroup	<th scope="col">
<td>	colspan, rowspan	N de celdas a abarcar	colspan="2"
<i>Formularios</i>			
<form>	action	URL del handler	action="/api/users"
	method	GET, POST	method="POST"
	enctype	MIME para uploads	enctype="multipart/form-data"
<input>	type	text, email, password, number, date, tel, url, checkbox, radio, file, hidden, range, color	type="email"
	name	Nombre del campo	name="email"
	value	Valor inicial	value="default"
	required	Booleano: obligatorio	required
	pattern	Regex de validación	pattern="[0-9]{10}"
	min/max/step	Rango numérico/fecha	min="0"max="100"
	minlength/maxlength	Longitud texto	maxlength="80"
	autocomplete	on, off, name, email...	autocomplete="email"
<label>	for	ID del input asociado	<label for="em">
<select>	multiple	Booleano: múltiple selección	multiple
<option>	value, selected	Valor; preselección	<option value="EC"selected>
<textarea>	rows, cols	Tamaño visible	rows="5"
<button>	type	submit, button, reset	type="submit"

Esta tabla está organizada para servir como *referencia rápida* durante la práctica. Para profundizar en los atributos específicos de formularios (tipos de input, validación nativa, accesibilidad de controles) la semana 2 dedica una sección completa al tema.

1.7.1 Atributos globales (aplican a casi todos los tags)

Los atributos globales son aquellos que pueden aplicarse a *cualquier* elemento HTML5, no solo a unos pocos. Comprenderlos en profundidad evita reaprenderlos para cada nuevo tag. La tabla 1.5 cataloga los más importantes con su efecto y un ejemplo breve.

Tabla 1.5: Atributos HTML5 globales.

Atributo	Efecto / valores	Ejemplo
id	Identificador único en el documento	id="hero"
class	Una o varias clases CSS separadas por espacio	class="btn primario"
style	CSS inline (uso desaconsejado)	style="color:red"
title	Tooltip flotante al pasar el mouse	title="Cerrar"
lang	Idioma del contenido (ISO 639-1 y 3166-1)	lang="es-EC"
dir	Dirección texto: ltr, rtl, auto	dir="rtl"
hidden	Booleano: oculta el elemento	hidden
tabindex	Orden de tabulación; 0 (natural), -1 (programático)	tabindex="0"
role	Rol ARIA explícito	role="navigation"
aria-label	Etiqueta accesible cuando no hay texto visible	aria-label="Cerrar"
aria-hidden	true/false: oculta para lectores	aria-hidden="true"
data-*	Datos custom para JavaScript	data-id="42"
contenteditable	true/false	contenteditable="true"
draggable	Booleano: arrastrable	draggable="true"

Ejemplo corto comentado (atributos globales en acción) (Listado 1.3):

Listing 1.3: Ejemplo de HTML

```

<!-- 'id' identifica univocamente al elemento (uno por documento) -->
<!-- 'class' agrupa elementos para aplicar el mismo estilo CSS -->
<!-- 'title' muestra un tooltip al pasar el cursor -->
<!-- 'lang' especifica el idioma del contenido (importante a11y) -->
<!-- 'data-*' guarda datos personalizados accesibles desde JS -->
<article id="post-42" class="card_destacado"
        title="Noticia_destacada" lang="es"
        data-autor="ana" data-categoria="tecnologia">
    Contenido del artículo
</article>

```

1.7.2 Tags estructurales y de contenido

HTML5 introduce un conjunto de tags semánticos pensados para describir el *significado* del contenido, no solo su apariencia. La tabla 1.4 cataloga los tags estructurales y de contenido más usados, con sus atributos específicos y un ejemplo de uso para cada uno.

Ejemplo corto comentado (estructura semántica completa) (Listado 1.4):

Listing 1.4: Ejemplo de HTML

```

<!-- <header> contiene la cabecera del sitio o de una seccion -->
<header>
  <!-- <nav> agrupa enlaces de navegacion principal -->
  <nav>
    <a href="/inicio">Inicio</a>
    <a href="/contacto">Contacto</a>
  </nav>
</header>
<!-- <main> envuelve el contenido unico de la pagina (uno por doc) -->
<main>
  <!-- <article> es contenido autonomo (post, noticia, comentario) -->
  <article>

```

```
<!-- <h1>...<h6> jerarquia de encabezados (h1 = mas importante) -->
<h1>Titulo del articulo</h1>
<!-- <section> agrupa contenido tematicamente relacionado -->
<section>
  <p>Parrafo de texto en la seccion.</p>
</section>
</article>
</main>
<!-- <footer> contiene pie de pagina (copyright, contacto, sitemap) -->
<footer><p>(c) 2026 UTEQ</p></footer>
```